Analytics Engineering: Session 5

Practice Session I: Building the Foundation

Goal

Build a complete DAG to understand our orders.

We should be able to answer:

- How many orders were placed each day?
- Are our orders late? By how much?
- Which of the carriers performs best?
- What is the total revenue per customer segment and month?
- Which products are the most popular?

Checklist

1. Complete all staging models (one per source table)
2. Add `unique` and `not_null` tests to all staging primary keys
3. Build intermediate models
4. Build mart models (dimensions and facts)
5. Add generic tests (`unique`, `not_null`, `relationships`) to all models
6. Run `dbt build` and verify everything passes
7. Verify the lineage graph

The DAG: Intermediate Layer

Goal: Prepare data for joining and isolate complexity.

We don't want to join raw tables directly into our final Marts if the logic is complex or reusable.

Model 1: `int_order_items_summary`

The Problem: `orders` has 1 row per order. `order_items` has N rows per order. Joining them directly causes "fan-out" (duplication of order value).

The Work:

1. Select from `stg_order_items`.
2. `GROUP BY order_id`.
3. `SUM(quantity)`, `SUM(total_price)`, `COUNT(item_id)`.

Why: We create a 1:1 relation with `orders`, making downstream joins safe and simple.

Model 2: `int_order_shipping`

The Problem: Shipping metrics require dates from both `orders` (placed date) and `shipping` (shipped date).

The Work:

1. Join `stg_orders` + `stg_shipping`.
2. Calculate `days_to_ship` (`ship_date` - `order_date`).
3. Calculate `is_late` (`actual_delivery` > `estimated_delivery`).

Why: Centralize the definition of "Late". If the business changes the definition (e.g., "Late means > 2 days after estimate"), we update it in one place.

The DAG: Marts Layer (The Fact Table)

Goal: The central source of truth.

Model 3: `mart_orders`

The Problem: Analysts need one table with *everything* about an order (status, items, shipping, payment). They shouldn't have to join 5 tables every time.

The Work:

1. Select from `stg_orders`.
2. Join `int_order_items_summary` (for item metrics).
3. Join `int_order_shipping` (for shipping metrics).

Why: Creates a "Wide Table" that is easy to query in BI tools (Tableau/Looker) without further joins.

Adding Tests to `mart_orders`

```yaml
models:
  - name: mart_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

The DAG: Marts Layer (Analysis)

Goal: Answer specific business questions.

Model 4: `mart_revenue_by_segment`

The Problem: Executives want to know which customer segments drive the most revenue, but `mart_orders` is too granular.

The Work:

1. Join `mart_orders` + `dim_customers`.
2. `GROUP BY customer_segment`.
3. `SUM(total_amount)`, `AVG(total_amount)`.

Why: Provides a high-level strategic view for decision-making (e.g., "Should we target Gold customers more?").

Model 5: `mart_product_performance`

The Problem: Merchandising teams need to know which products are selling best.

The Work:

1. Join `stg_order_items` + `dim_products`.
2. `GROUP BY product_id, category_name`.
3. `SUM(revenue)`, `SUM(quantity)`.

Why: Enables inventory planning and marketing focus on high-performing items.

Model 6: `customers_enriched_python`

The Problem: We need to categorize customers by email provider (Gmail, Yahoo, etc.), but SQL string parsing can be messy.

The Work:

1. Read `dim_customers`.
2. Use **Polars** (Python) to split email strings and map domains.
3. Return a DataFrame.

Why: Python is superior for complex text processing and allows us to use libraries like Polars for performance.

Final Verification

Run the complete build and verify:

```
dbt build
```

- All models should compile and run successfully
- All tests should pass (unique, not_null, relationships)
- Check the lineage graph — it should flow cleanly left-to-right
- Run `dbt source freshness` and interpret results

What have we learned in this session

- Built intermediate models: `int_order_items_summary`, `int_order_shipping`
- Created mart models: `mart_orders`, `dim_customers`, `dim_products`
- Added generic tests across all layers
- Used `dbt build` to run models and tests together
- Verified the full DAG with lineage graph

Next Session: Materializations & Grants.